

Senior Frontend Interview Questions & Answers

JavaScript

Event Loop

Q1: What is the Event Loop?

Answer: It coordinates execution between the call stack, Web APIs, microtask queue, and task queue.

Q2: Microtasks vs Macrotasks?

Answer: Promises run before setTimeout callbacks after the current stack is empty.

Q3: Output of Promise + setTimeout example?

Answer: Microtasks execute before timers.

Promises

Q1: What problem do Promises solve?

Answer: Callback hell and async flow management.

Q2: Promise.all vs Promise.allSettled?

Answer: all fails fast; allSettled waits for every promise.

Q3: When would you use Promise.race?

Answer: Timeouts and competing async operations.

Async/Await

Q1: How does async/await work?

Answer: Syntactic sugar over Promises.

Q2: Can await block the UI thread?

Answer: No, it pauses only the async function.

Q3: How do you handle errors?

Answer: try/catch or promise chaining.

Closures

Q1: What is a closure?

Answer: A function retaining access to outer scope variables.

Q2: Real-world use?

Answer: Private state, factories, debounce implementations.

Q3: Potential downside?

Answer: Can accidentally retain memory.

Hoisting

Q1: Difference between var and let hoisting?

Answer: var is initialized as undefined; let/const are in the TDZ.

Q2: Are functions hoisted?

Answer: Function declarations are fully hoisted.

Prototypes

Q1: What is the prototype chain?

Answer: Inheritance mechanism used by JavaScript objects.

Q2: Class vs prototype?

Answer: Classes are syntactic sugar over prototypes.

Memory Leaks

Q1: Common memory leak causes?

Answer: Detached DOM nodes, timers, event listeners, global references.

Q2: How to detect leaks?

Answer: Chrome DevTools heap snapshots.

Debounce vs Throttle

Q1: Debounce?

Answer: Executes after activity stops.

Q2: Throttle?

Answer: Limits execution frequency.

Browser Rendering Pipeline

Q1: Rendering stages?

Answer: Parse HTML → DOM/CSSOM → Render Tree → Layout → Paint → Composite.

Q2: Reflow vs Repaint?

Answer: Layout changes vs visual changes.

Frontend Architecture

Component Design

Q1: Smart vs Dumb components?

Answer: Container logic separated from presentation.

Q2: How do you build reusable components?

Answer: Clear APIs, composition, isolation.

State Management

Q1: Local vs Global state?

Answer: Component-specific versus application-wide.

Q2: When would Pinia/Zustand be needed?

Answer: Shared complex state.

Data Fetching

Q1: SSR vs CSR?

Answer: Server-side rendering vs client-side rendering.

Q2: Stale-while-revalidate?

Answer: Serve cache first, refresh in background.

Caching

Q1: Client caching strategies?

Answer: Memory, localStorage, IndexedDB, service workers.

Error Handling

Q1: Frontend error handling?

Answer: Graceful fallbacks, retry mechanisms, logging.

Feature Ownership

Q1: What does ownership mean?

Answer: Responsible from design to monitoring.

Performance

Virtualization

Q1: Why virtualize lists?

Answer: Render only visible items.

Lazy Loading

Q1: What can be lazy loaded?

Answer: Images, routes, components, scripts.

Code Splitting

Q1: Benefits?

Answer: Smaller initial bundles and faster load times.

Bundle Optimization

Q1: Optimization techniques?

Answer: Tree shaking, minification, dynamic imports.

Re-render Optimization

Q1: How to reduce re-renders?

Answer: Memoization, stable references, component splitting.

Web Vitals

Q1: Core Web Vitals?

Answer: LCP, INP, CLS.

System Design

Real-Time Analytics Dashboard

Q1: How would you handle live updates?

Answer: WebSockets or SSE.

Q2: How would you scale charts?

Answer: Aggregation and windowing.

Q3: Caching strategy?

Answer: Client cache plus backend cache layer.

Q4: Pagination or infinite scroll?

Answer: Depends on analytics workflow; often server-side pagination.

Q5: Error recovery?

Answer: Reconnect logic, retries, offline state.

Q6: Performance concerns?

Answer: Virtualization, memoization, selective updates.